

Primeiro Exercício -Programa
Escalonador de Processos para Time Sharing
Sistemas Operacionais

João Pedro Becer — N^o USP 16987046

Outubro de 2026

Resumo

Este relatório descreve a implementação de um escalonador de processos por prioridades com créditos para uma máquina fictícia de um único processador e avalia o comportamento do sistema para doze valores de quantum (de 1 a 21 instruções). São analisados o número médio de trocas por processo e o número médio de instruções executadas por quantum. A análise dos logs mostra que, independentemente do quantum, 23 das trocas do sistema são causadas por E/S e 10 pelo término dos processos, e que apenas dois dos dez programas possuem surtos de CPU longos o bastante para serem afetados por quanta acima de 11. Com base em dois critérios independentes (custo composto normalizado e ganho marginal), conclui-se que o quantum mais adequado é $n = 8$.

Sumário

1	Introdução	4
2	Estrutura do Projeto	4
2.1	Organização dos arquivos	4
2.2	Bloco de Controle de Processo (BCP)	4
2.3	Tabela de processos e filas	5
2.4	Leitor	5
2.5	Escritor	5
2.6	Gerenciador de processos e escalonador	6
2.7	Decisões de projeto	6
3	Metodologia	7
3.1	Ambiente de teste	7
3.2	Valores de quantum testados	7
3.3	Métricas analisadas	7
4	Resultados	8
4.1	Dados coletados	8
4.2	Análise gráfica	9
5	Análise dos resultados	10
5.1	Por que as curvas têm esse formato	10
5.2	Quantum baixo ($n = 1$ a 4)	12
5.3	Quantum médio ($n = 6$ a 12)	12
5.4	Quantum alto ($n = 14$ a 21)	12
5.5	Descobertas	13
6	Determinação do quantum ótimo	13
6.1	Critérios de avaliação	13
6.2	Análise quantitativa	14
6.3	Justificativa da escolha	15
7	Conclusões	15

1 Introdução

Em um sistema de multiprogramação com compartilhamento de tempo, o escalonador decide qual processo ocupa o processador a cada instante. Quanto menor a fatia de tempo (quantum) concedida a cada processo, mais responsivo o sistema parece, mas maior o número de trocas de contexto, que são puro custo de gerenciamento. Quanto maior o quantum, menos trocas ocorrem, mas processos que usam muito a CPU passam a atrasar os demais. A escolha do quantum é, portanto, um compromisso.

Neste trabalho foi implementado um escalonador para uma máquina fictícia com apenas dois registradores de uso geral (X e Y), além do contador de programa, e quatro instruções: atribuição ($X=v$ ou $Y=v$), entrada e saída (E/S), comando genérico (COM) e fim de programa (SAIDA). Como não há *clock* para preempção, o próprio escalonador funciona como interpretador: lê e executa cada instrução, e o quantum é medido em número de instruções. O algoritmo de escalonamento usa créditos proporcionais à prioridade, de forma semelhante ao escalonador do Linux.

Os objetivos são:

1. implementar o escalonador conforme a especificação, com BCP, tabela de processos, fila de prontos e fila de bloqueados explícitos;
2. gerar os logs de execução para diferentes valores de quantum;
3. analisar o número médio de trocas e o número médio de instruções por quantum;
4. determinar, com fundamento nos dados, o quantum mais adequado.

O restante do documento está organizado assim: a Seção 2 descreve a estrutura do projeto; a Seção 3, a metodologia; a Seção 4 apresenta os dados coletados; a Seção 5 analisa o comportamento observado; a Seção 6 determina o quantum ótimo; e a Seção 7 traz as conclusões.

2 Estrutura do Projeto

2.1 Organização dos arquivos

O sistema foi escrito em C++17 e dividido em três módulos, compilados por um `makefile` (`g++ -std=c++17 -Wall -Wextra`):

- `main.cpp`: carrega os processos, executa o escalonador e fecha o log;
- `escalonador.hpp/.cpp`: BCP, classe `Process`, tabela de processos, filas e o laço de escalonamento;
- `utils.hpp/.cpp`: leitura de `quantum.txt` e `prioridades.txt`, enumeração de estados e funções de escrita do log.

2.2 Bloco de Controle de Processo (BCP)

O BCP é a estrutura BCP, que contém toda a informação necessária para retomar um processo interrompido (Tabela 1). A prioridade fica na classe `Process`, que encapsula o

BCP, e o campo `creditos` é o contador que decresce durante a execução e é restaurado à prioridade na redistribuição.

Tabela 1: Campos do BCP e da classe `Process`.

Campo	Função
PC	Contador de programa: próxima instrução a executar
estado	executando, pronto, bloqueado ou terminado
X, Y	Registradores de uso geral (iniciam em zero)
creditos	Créditos restantes do processo
quantum	Tamanho do quantum (lido de <code>quantum.txt</code>)
tempEspera	Tempo de espera restante, quando bloqueado por E/S
nomePrograma	Nome do programa (primeira linha do arquivo)
codigo	Segmento de texto: um <code>vector<string></code> , uma instrução por posição
<code>Process::prioridade</code>	Prioridade, lida de <code>prioridades.txt</code>
<code>Process::id</code>	Número do arquivo (1 a 10), usado como desempate final
<code>Process::ultimaExecucao</code>	Carimbo da última vez em que foi escalonado (desempate)

2.3 Tabela de processos e filas

- **Tabela de processos:** `std::vector<Process>`, com todos os processos do sistema. A capacidade é reservada antes das inserções, de modo que os ponteiros usados nas filas permanecem válidos.
- **Fila de prontos:** `std::priority_queue<Process*>` com o comparador `comparaCreditos`, que mantém no topo o processo de maior número de créditos.
- **Fila de bloqueados:** `std::queue<Process*>`, em ordem de chegada. Como o tempo de resposta da E/S é igual para todos, ela se comporta como fila comum.

2.4 Leitor

As funções `getQuantum()` e `getPrioridade(linha)`, em `utils.cpp`, leem, respectivamente, o inteiro de `programas/quantum.txt` e a prioridade da linha correspondente de `programas/prioridades.txt`. A função `inicializarProcessos()` abre `01.txt`, `02.txt`, ..., lê o nome do programa (primeira linha) e as instruções (demais linhas), cria o BCP com `PC = 0` e créditos iguais à prioridade, e insere o processo na tabela e na fila de prontos. Ao final, o log registra os nomes carregados *na ordem da fila de prontos*, isto é, por prioridade, como pede a especificação.

2.5 Escritor

O módulo de log (`inicializarLog`, `logCarregando`, `logExecutando`, `logInterrompendo`, `logES`, `logTerminado`, `logEstatisticas` e `fecharLog`) grava o arquivo `logXX.txt`, em que `XX` é o quantum com dois dígitos. Cada decisão de escalonamento (carregar,

executar, interromper, iniciar E/S e terminar) gera uma linha, e as estatísticas finais são escritas ao término da execução. Manter a escrita isolada em `utils.cpp` separa a lógica de escalonamento do formato de saída.

2.6 Gerenciador de processos e escalonador

A função `executarProcessos()` implementa o algoritmo de créditos. A cada iteração do laço principal:

1. Se não há processos prontos, decrementa-se o tempo de espera de todos os bloqueados até que algum fique pronto.
2. Retira-se do topo da fila o processo de maior crédito, marca-se como **executando**, decrementa-se um crédito e registra-se *Executando* no log.
3. O processo interpreta até n instruções. A execução termina antes se encontrar E/S (o processo é bloqueado com `tempEspera = 2`) ou SAIDA (o processo termina). E/S e SAIDA contam como instruções executadas, e as três saídas do quantum (E/S, fim de quantum e término) contam como troca nas estatísticas.
4. Se esgotou o quantum sem bloquear nem terminar, o processo volta à fila de prontos, na posição dada pelos créditos restantes.
5. Ao final do quantum, o tempo de espera de todos os bloqueados é decrementado; os que chegam a zero vão para a fila de prontos.
6. Por fim, `redistribuirCreditos()` verifica se todos os processos ainda não terminados estão com zero crédito. Em caso afirmativo, restaura os créditos à prioridade e reconstrói a fila, pois a ordem do *heap* fica inválida quando os créditos mudam.

2.7 Decisões de projeto

Desempate entre processos com o mesmo número de créditos. A especificação pede que se analise o caso em que o processo que acabou de rodar tem, após o desconto do crédito, o mesmo número de créditos que o segundo da fila. Em um sistema real, uma troca completa de contexto é mais custosa que manter o processo corrente: exige salvar e restaurar registradores e perde-se a localidade de cache e da TLB. Por isso, o comparador usa três critérios, nesta ordem: (i) mais créditos; (ii) *execução mais recente*, que mantém o processo que já estava na CPU; (iii) menor `id`, que torna a ordem determinística. No simulador o custo é idêntico, mas a política escolhida é a que seria melhor em uma máquina real. Os pares com a mesma prioridade (TESTE-8 e TESTE-9, ambos com 8; TESTE-5 e TESTE-10, ambos com 4) são desempatados pelo `id`, o que se reflete na ordem de carregamento dos logs.

```
bool operator()(const Process* a, const Process* b) const{
    if(a->bcp.creditos != b->bcp.creditos)
        return a->bcp.creditos < b->bcp.creditos;
    if(a->ultimaExecucao != b->ultimaExecucao)
        return a->ultimaExecucao < b->ultimaExecucao;
    return a->id > b->id;
}
```

Tempo de espera da E/S. O processo bloqueado é inserido na fila de bloqueados *depois* do decremento das esperas daquele quantum. Assim ele não decrementa o próprio tempo, e permanece bloqueado até que **dois outros processos** passem pelo estado **executando**, como determina a especificação. Isso pode ser verificado nos logs: por exemplo, em `log20.txt`, após a E/S de TESTE-7 executam TESTE-4 e TESTE-8 antes de TESTE-7 voltar.

Processos terminados. Um processo que executa **SAIDA** é marcado como **terminado** e não volta a nenhuma fila. Ele permanece no vetor da tabela de processos, para manter válidos os ponteiros das filas, mas é ignorado na redistribuição de créditos. Sem esse cuidado, um processo terminado com crédito remanescente impediria a redistribuição indefinidamente.

3 Metodologia

3.1 Ambiente de teste

Os testes foram executados em um computador com Linux, usando o compilador GCC (`g++ -std=c++17 -Wall -Wextra`). Foram usados os 10 programas fornecidos (`01.txt` a `10.txt`), com os arquivos `prioridades.txt` e `quantum.txt` originais; apenas o quantum foi alterado entre as execuções. Como o escalonador é determinístico, cada configuração precisa ser executada uma única vez.

As prioridades, lidas de `prioridades.txt`, são mostradas na Tabela 3. A ordem de carregamento registrada em todos os logs (TESTE-7, 4, 8, 9, 6, 5, 10, 2, 3, 1) corresponde à ordem decrescente de prioridade.

3.2 Valores de quantum testados

Foram testados doze valores: $n \in \{1, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 21\}$. Os dez valores de 2 a 20, com passo 2, cobrem o intervalo de forma uniforme. O valor $n = 1$ foi incluído como extremo inferior e $n = 21$ como extremo superior: 21 é o tamanho máximo de um programa, e acima disso nenhum processo esgota o quantum, de modo que os resultados não mudam.

3.3 Métricas analisadas

Seja $N = 10$ o número de processos. São coletadas:

- **Média de trocas** (M_T): número total de vezes em que um processo deixa a CPU, por E/S, por fim de quantum ou por término (**SAIDA**), dividido por N . O término é contado porque o enunciado inclui, na média de trocas, a última vez em que cada processo rodou.
- **Média de instruções por quantum** (M_I): total de instruções executadas (incluindo E/S e **SAIDA**) dividido pelo número total de quanta, isto é, de vezes em que algum processo foi colocado para executar.

Identidades úteis. Seja $I = 139$ o total de instruções dos 10 programas, valor que independe do quantum. Seja E o número de interrupções por E/S e $F(n)$ o número de interrupções por fim de quantum. Cada execução termina em interrupção (por E/S ou por fim de quantum) ou em término, e cada processo termina uma única vez. Logo

$$\begin{aligned} \text{trocas} = \text{quanta} &= E + F(n) + N, \\ M_T &= \frac{E + F(n) + N}{N}, \quad M_I = \frac{I}{E + F(n) + N} = \frac{I}{N M_T}. \end{aligned} \quad (1)$$

Em particular, $M_T \cdot M_I = I/N = 13,9$ para todo n : as duas métricas são a mesma informação vista de lados opostos, e o que as distingue é o significado de cada uma (custo de troca e aproveitamento de cada surto de CPU).

Chamamos de *surto de CPU* o trecho de instruções que um processo executa entre dois bloqueios (ou entre o último bloqueio e o término). Um surto de b instruções gera $\lceil b/n \rceil - 1$ interrupções por fim de quantum, portanto $F(n) = \sum_b (\lceil b/n \rceil - 1)$.

4 Resultados

4.1 Dados coletados

A Tabela 2 resume os logs gerados (`log01.txt` a `log21.txt`). O total de trocas foi conferido contando as linhas dos logs, e coincide com as médias reportadas.

Tabela 2: Métricas obtidas para cada quantum.

n	Trocas totais (= quanta)	Média de trocas M_T	Média de instr. por quantum M_I	% do máximo
1	139	13,9	1,00000	23,7
2	79	7,9	1,75949	41,8
4	52	5,2	2,67308	63,5
6	41	4,1	3,39024	80,5
8	38	3,8	3,65789	86,8
10	37	3,7	3,75676	89,2
12	35	3,5	3,97143	94,3
14	34	3,4	4,08824	97,1
16	34	3,4	4,08824	97,1
18	34	3,4	4,08824	97,1
20	34	3,4	4,08824	97,1
21	33	3,3	4,21212	100,0

4.2 Análise gráfica

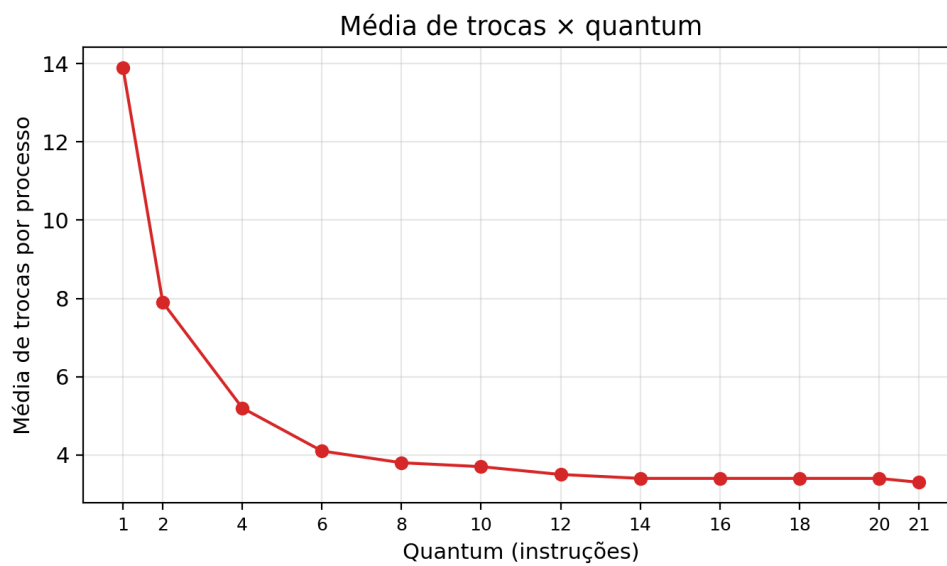


Figura 1: Média de trocas por processo em função do quantum.

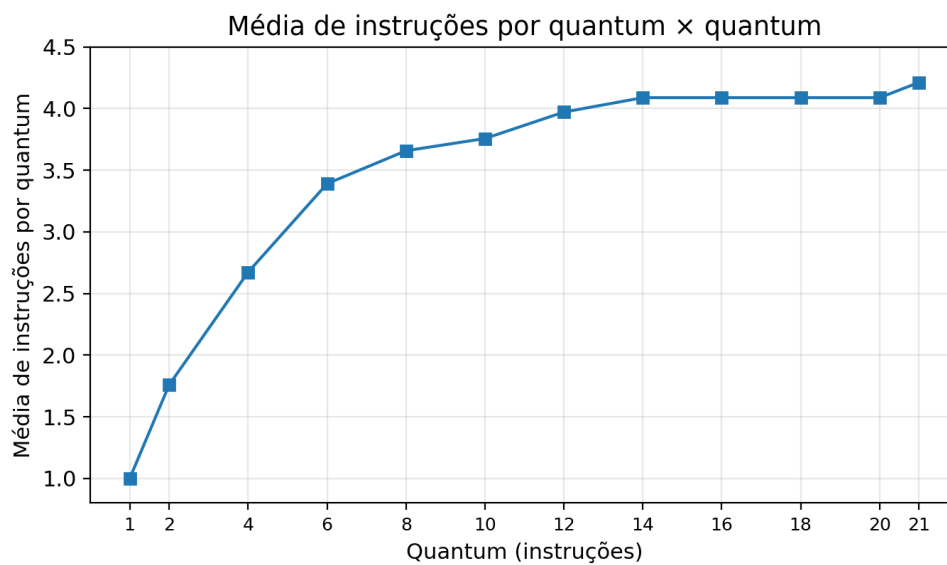


Figura 2: Média de instruções por quantum em função do quantum.

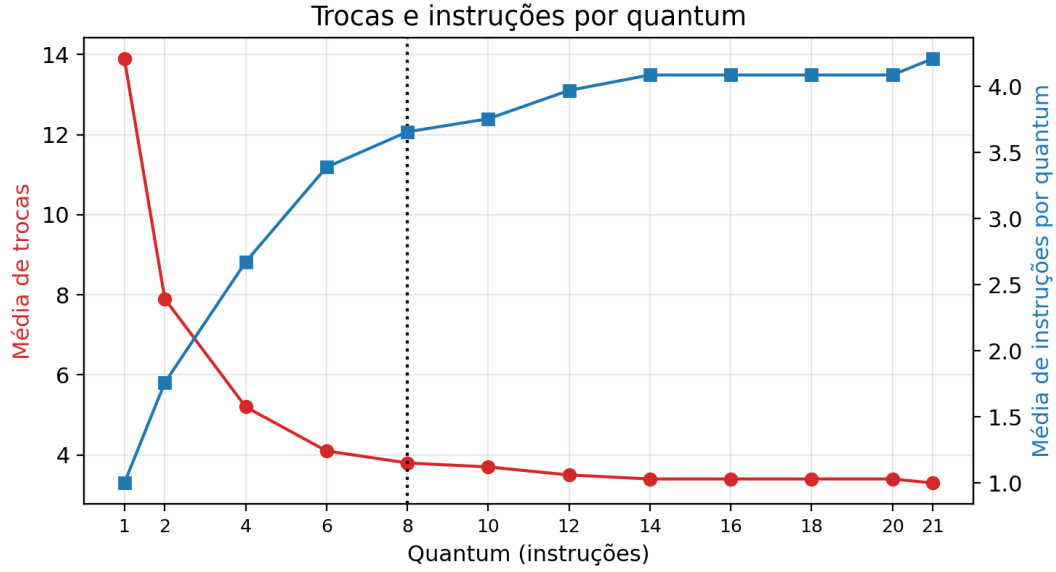


Figura 3: As duas métricas no mesmo gráfico. A linha pontilhada marca $n = 8$.

A Figura 1 mostra queda acentuada de M_T até $n \approx 6$, desaceleração entre 6 e 12 e estabilização em 3,4 de $n = 14$ a $n = 20$, com um último pequeno ganho em $n = 21$ (3,3). A Figura 2 mostra o comportamento simétrico de M_I , que cresce de 1,0 a 4,21 e tem o mesmo patamar entre 14 e 20. A Figura 3 evidencia que as duas curvas têm o mesmo “joelho”: a região em que as duas melhoram rapidamente termina por volta de $n = 6$ a 8.

5 Análise dos resultados

5.1 Por que as curvas têm esse formato

Os 10 programas totalizam 139 instruções e possuem 23 instruções de E/S, que formam 33 surtos de CPU (23 terminados por E/S e 10 por SAIDA). A Tabela 3 traz os dados de cada programa, obtidos de `log01.txt`, em que cada execução corresponde a uma instrução.

Tabela 3: Características dos programas (instruções incluem SAIDA; surtos em instruções).

Programa	Prioridade	Instruções	E/S	Surtos de CPU
TESTE-1	1	10	2	5, 4, 1
TESTE-2	3	21	3	5, 5, 5, 6
TESTE-3	2	14	0	14
TESTE-4	10	12	5	2, 2, 2, 2, 3, 1
TESTE-5	4	11	5	2, 2, 2, 2, 2, 1
TESTE-6	6	15	3	4, 7, 3, 1
TESTE-7	12	14	2	4, 9, 1
TESTE-8	8	21	0	21
TESTE-9	8	15	2	3, 11, 1
TESTE-10	4	6	1	5, 1
Total		139	23	33 surtos

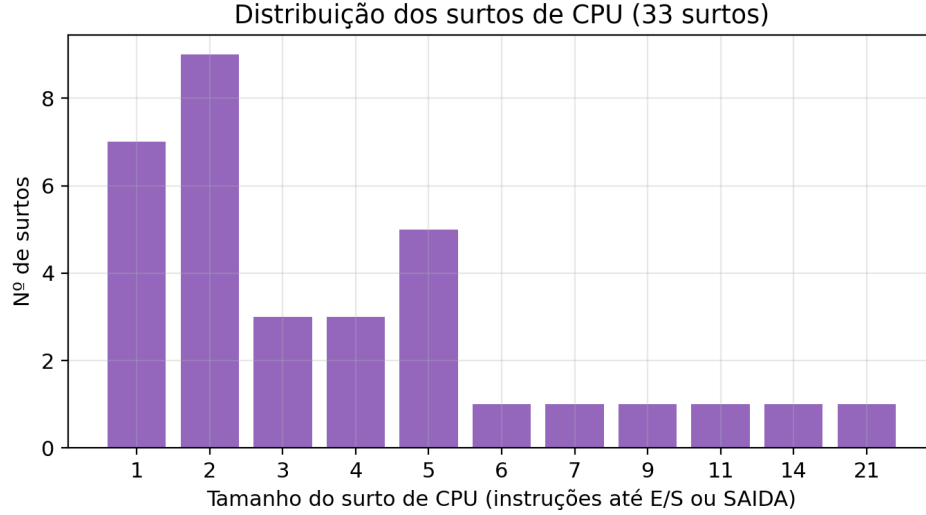


Figura 4: Distribuição dos 33 surtos de CPU. A maioria é curta; apenas TESTE-3 (14) e TESTE-8 (21) passam de 11 instruções.

Os dados mostram que o conjunto é dominado por processos limitados por E/S: o surto médio tem $139/33 \approx 4,2$ instruções, a mediana é 3, e 31 dos 33 surtos têm no máximo 11 instruções. Apenas TESTE-3 e TESTE-8, que não fazem E/S, têm surtos longos (14 e 21).

A decomposição das trocas (Figura 5 e Tabela 4) esclarece o resultado. As 23 interrupções por E/S e os 10 términos ocorrem em todos os quanta testados, e portanto $E + N = 33$ é um piso que nenhum valor de quantum elimina. Somente $F(n)$ varia: cai de 106, com $n = 1$, para 1 de $n = 14$ a $n = 20$, e para 0 com $n = 21$.

Tabela 4: Decomposição das trocas. Verifica-se $E + F + N = \text{trocas totais} = \text{quanta}$.

Quantum	Por E/S (E)	Por fim de quantum (F)	Por término (N)	Total
1	23	106	10	139
2	23	46	10	79
4	23	19	10	52
6	23	8	10	41
8	23	5	10	38
10	23	4	10	37
12	23	2	10	35
14	23	1	10	34
16	23	1	10	34
18	23	1	10	34
20	23	1	10	34
21	23	0	10	33

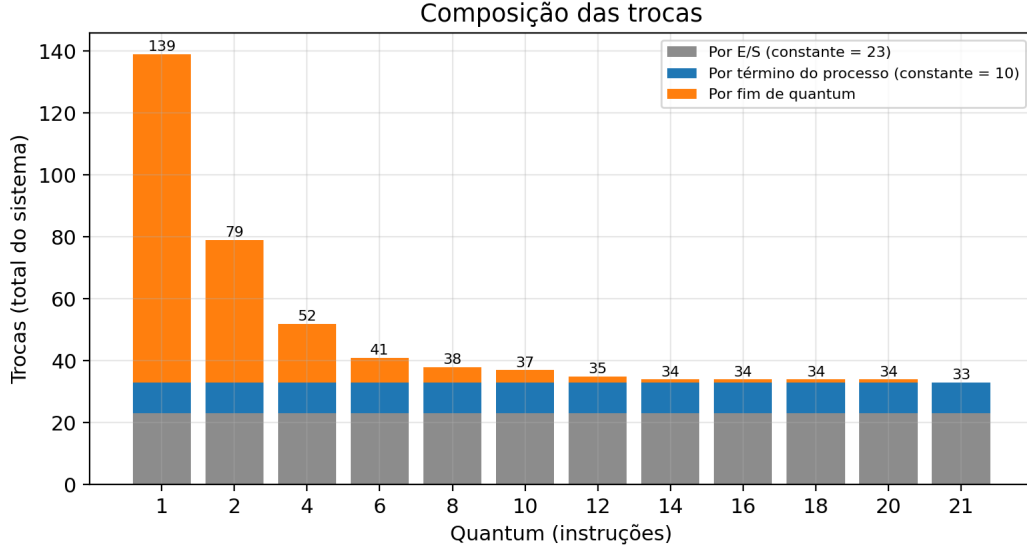


Figura 5: Composição do total de trocas: duas parcelas fixas (E/S e término) e uma parcela variável (fim de quantum).

O valor residual $F = 1$ para $14 \leq n \leq 20$ vem de TESTE-8, cujo surto de 21 instruções só cabe em um quantum inteiro com $n \geq 21$. Isso é confirmado por `log21.txt`: nele TESTE-8 executa as 21 instruções de uma vez e termina, e as estatísticas são $M_T = 3,3$ e $M_I = 4,21212$, exatamente os valores previstos pela Equação (1) com $F = 0$. A mesma fórmula explica o ponto $n = 12$: nele são interrompidos o surto de 14 (TESTE-3) e o de 21 (TESTE-8), logo $F = 2$.

5.2 Quantum baixo ($n = 1$ a 4)

Com $n = 1$ cada instrução é um quantum: `log01.txt` traz 139 execuções, das quais 129 terminam em interrupção e 10 em término. Cada processo deixa a CPU, em média, 13,9 vezes, e a CPU não executa mais de uma instrução por escalonamento. Com $n = 2$ e $n = 4$ o cenário melhora, mas n ainda é menor que o surto médio (4,2), de modo que até surtos típicos são fragmentados: M_I fica em 42% e 64% do máximo. A redução do total de trocas em cada passo é grande (43,2% de 1 para 2 e 34,2% de 2 para 4), o que mostra que nessa região o sistema está dominado pelo custo de troca.

5.3 Quantum médio ($n = 6$ a 12)

A partir de $n = 6$ o quantum já supera o surto médio, e F cai de 19 para 8, 5, 4 e 2. A Figura 3 mostra o “joelho” nessa faixa. O ganho de trocas totais entre passos consecutivos cai para 21,2% (4 para 6), 7,3% (6 para 8), 2,6% (8 para 10) e 5,4% (10 para 12): em termos absolutos, de 3, 1 e 2 trocas, em um total de cerca de 40. Como exemplo, em `log08.txt` o programa TESTE-8 (21 instruções) é dividido em três execuções (8, 8 e 5), de modo que os processos prontos não esperam mais de 8 instruções por causa dele.

5.4 Quantum alto ($n = 14$ a 21)

Para $14 \leq n \leq 20$ os resultados são idênticos ($M_T = 3,4$ e $M_I = 4,09$): cada processo executa até bloquear ou terminar, e o quantum só age sobre TESTE-8. Com $n = 21$ há

um último ganho de uma troca ($M_T = 3,3$, $M_I = 4,21$), pois nenhum processo é mais interrompido por fim de quantum. Aumentar n além de 14 traz, portanto, um benefício mínimo, e tem custo: em `log20.txt` TESTE-8 ocupa a CPU por 20 instruções seguidas, atrasando todos os processos prontos, e em `log21.txt` chega a 21. Em um sistema com processos interativos esse efeito prejudicaria o tempo de resposta, e no limite o escalonador se comportaria como uma política sem preempção.

5.5 Descobertas

- As duas métricas carregam a mesma informação: $M_T \cdot M_I = 13,9$ para todo n (Equação 1), e ambas dependem apenas de $F(n)$.
- Com $n = 8$, das 38 trocas, 23 (61%) são causadas por E/S e 10 (26%) pelo término dos processos; somente 5 (13%) dependem do quantum.
- O platô a partir de $n = 14$ se deve à distribuição de surtos, e não a um limite do algoritmo: acima de $n = 11$, somente os surtos de 14 e 21 instruções (TESTE-3 e TESTE-8) geram interrupções por fim de quantum.
- O resultado depende da carga: com programas mais limitados por CPU, o platô ocorreria em quanta maiores.

6 Determinação do quantum ótimo

6.1 Critérios de avaliação

Menos trocas e mais instruções por quantum são desejáveis, mas as duas métricas melhoram sempre que n cresce, de modo que sozinhas apontariam para $n = 21$. Falta o contrapeso: um quantum grande prejudica a responsividade e a justiça entre processos, pois um processo limitado por CPU pode ocupar o processador por n instruções seguidas. Foram usados dois critérios independentes.

Critério A: custo composto. Normalizam-se três custos para $[0, 1]$ (mínimo e máximo tomados sobre os valores testados, com $n_{\min} = 1$ e $n_{\max} = 21$):

$$J(n) = \underbrace{\frac{M_T(n) - M_T^{\min}}{M_T^{\max} - M_T^{\min}}}_{\text{custo de troca}} + \underbrace{\frac{M_I^{\max} - M_I(n)}{M_I^{\max} - M_I^{\min}}}_{\text{fragmentação}} + w \underbrace{\frac{n - n_{\min}}{n_{\max} - n_{\min}}}_{\text{latência}}, \quad (2)$$

com $w = 1$ (pesos iguais). O terceiro termo é uma medida simples do atraso imposto aos demais processos, proporcional ao tamanho do quantum.

Critério B: ganho marginal. Escolhe-se o menor n a partir do qual o próximo aumento reduz o total de trocas em menos de 10%, ou seja, o ponto em que o retorno passa a ser pequeno.

6.2 Análise quantitativa

Tabela 5: Termos do custo composto $J(n)$ (com $w = 1$) e redução marginal de trocas totais.

n	Troca	Fragment.	Latência	$J(n)$	Redução de trocas (vs. anterior)
1	1,000	1,000	0,000	2,000	—
2	0,434	0,764	0,050	1,248	43,2%
4	0,179	0,479	0,150	0,808	34,2%
6	0,075	0,256	0,250	0,581	21,2%
8	0,047	0,173	0,350	0,570	7,3%
10	0,038	0,142	0,450	0,630	2,6%
12	0,019	0,075	0,550	0,644	5,4%
14	0,009	0,039	0,650	0,698	2,9%
16	0,009	0,039	0,750	0,798	0,0%
18	0,009	0,039	0,850	0,898	0,0%
20	0,009	0,039	0,950	0,998	0,0%
21	0,000	0,000	1,000	1,000	2,9%

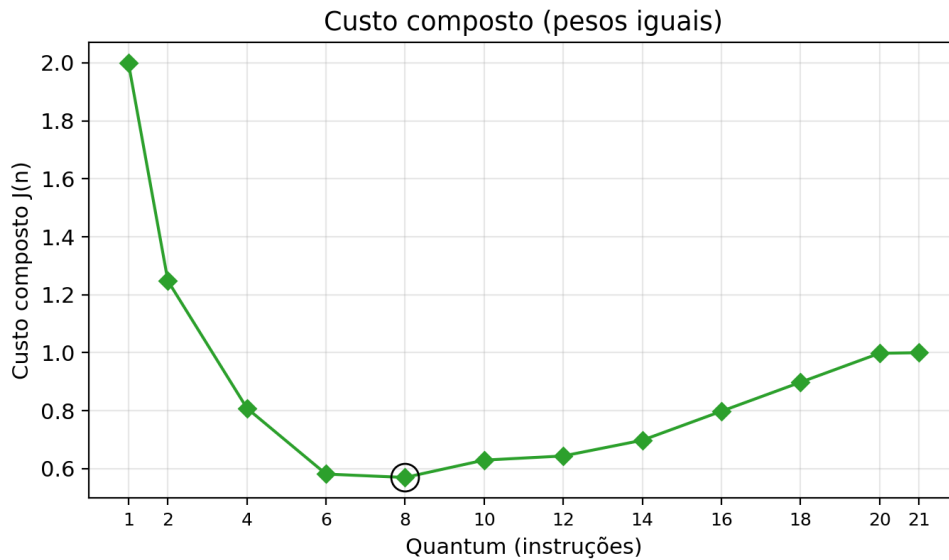


Figura 6: Custo composto $J(n)$ com pesos iguais. O mínimo ocorre em $n = 8$ (círculo).

No Critério A, o mínimo é $J(8) = 0,570$, seguido de perto por $J(6) = 0,581$ (diferença de cerca de 2%); $n = 10$ e $n = 12$ são piores porque o ganho em trocas é pequeno e a latência já cresceu. No Critério B, a primeira redução inferior a 10% é a de $n = 6$ para $n = 8$ (7,3%); todos os passos seguintes ficam abaixo de 10%, e o ganho acumulado de 8 até 21 é de apenas 5 trocas (13% das 38 trocas de $n = 8$).

Sensibilidade ao peso. Variando w de 0 a 3 em passos de 0,05, o mínimo de J é: $n = 21$ para $w < 0,15$; $n = 14$ para $0,15 \leq w < 0,50$; $n = 12$ para $0,50 \leq w < 0,65$; $n = 8$ para $0,65 \leq w < 1,15$; e $n = 6$ para $w \geq 1,15$. Portanto $n = 8$ é o ótimo em torno do peso neutro ($w = 1$), e a alternativa mais próxima é $n = 6$. Quanto mais se valoriza a

responsividade, menor deve ser o quantum; só quando a latência é praticamente ignorada ($w < 0,15$) o melhor valor é o máximo.

6.3 Justificativa da escolha

Adota-se $n = 8$. Com ele, a média de trocas é 3,8 (apenas 0,5 acima do mínimo observado, 3,3), a média de instruções por quantum é 3,66 (86,8% do máximo) e as trocas se reduzem em 72,7% em relação a $n = 1$. Das 38 trocas, apenas 5 são causadas pelo quantum; as outras 33 (E/S e término) não poderiam ser evitadas com nenhum valor de n . Aumentar o quantum para 14 ou mais eliminaria 4 dessas 5 interrupções (as 5, com $n = 21$), e em contrapartida permitiria a processos como TESTE-8 ocupar a CPU por até 14 a 21 instruções seguidas. Diminuir para 6 pioraria M_T em 0,3 e M_I em 0,27, com ganho de latência pequeno.

7 Conclusões

Foi implementado um escalonador de processos por créditos com BCP, tabela de processos, fila de prontos ordenada e fila de bloqueados, cujos logs seguem o formato especificado. Foram avaliados doze valores de quantum, de 1 a 21 instruções. As principais conclusões são:

1. Para quanta baixos ($n \leq 4$), o sistema é dominado pelo custo de troca: com $n = 1$ são 13,9 trocas por processo e apenas uma instrução por escalonamento.
2. Para quanta médios ($6 \leq n \leq 12$), os ganhos diminuem rapidamente, e o “joelho” das duas curvas está entre 6 e 8.
3. Para quanta altos ($14 \leq n \leq 20$), as métricas estabilizam em $M_T = 3,4$ e $M_I = 4,09$, porque quase todos os surtos de CPU são encerrados por E/S antes do fim do quantum. Com $n = 21$ há um último ganho (3,3 e 4,21), mas aumentar n não traz benefício relevante e apenas atrasa os demais processos.
4. Com $n \geq 8$, de 60% a 70% das trocas (23 de 33 a 38) são causadas por E/S e não dependem do quantum, e mais 10 são termos; isso limita o benefício de quanta grandes.
5. O quantum mais adequado para esta carga é **$n = 8$** , escolhido por dois critérios independentes (custo composto e ganho marginal) e robusto a variações moderadas do peso da latência ($0,65 \leq w < 1,15$).

Limitações. A conclusão se baseia em uma única carga de 10 programas, com E/S de duração fixa. A métrica de latência é uma aproximação (proporcional a n), e o valor ótimo pode mudar se o conjunto de programas tiver mais processos limitados por CPU.